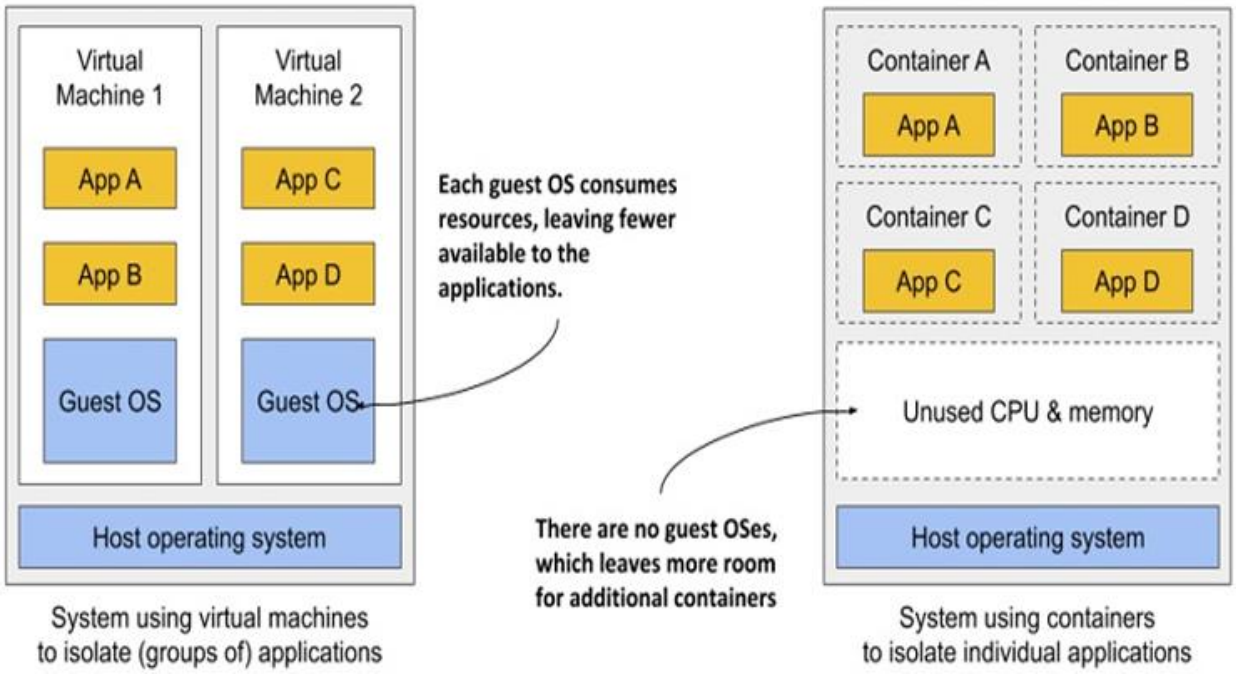
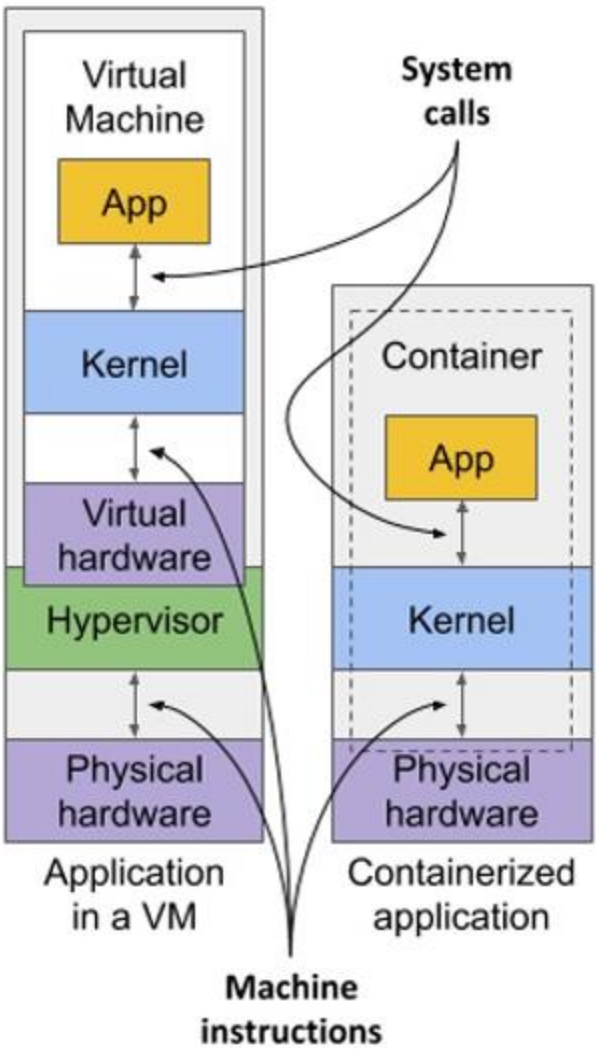
Unlike VMs, **which each run a separate operating system** with several system processes, a process running **in a container runs within the existing host operating system**. Because there is only one operating system, **no duplicate system processes exist**. Although all the application processes run in the same operating system, their environments are isolated, though not as well as when you run them in separate VMs. To the process in the container, this isolation makes it look like no other processes exist on the computer.

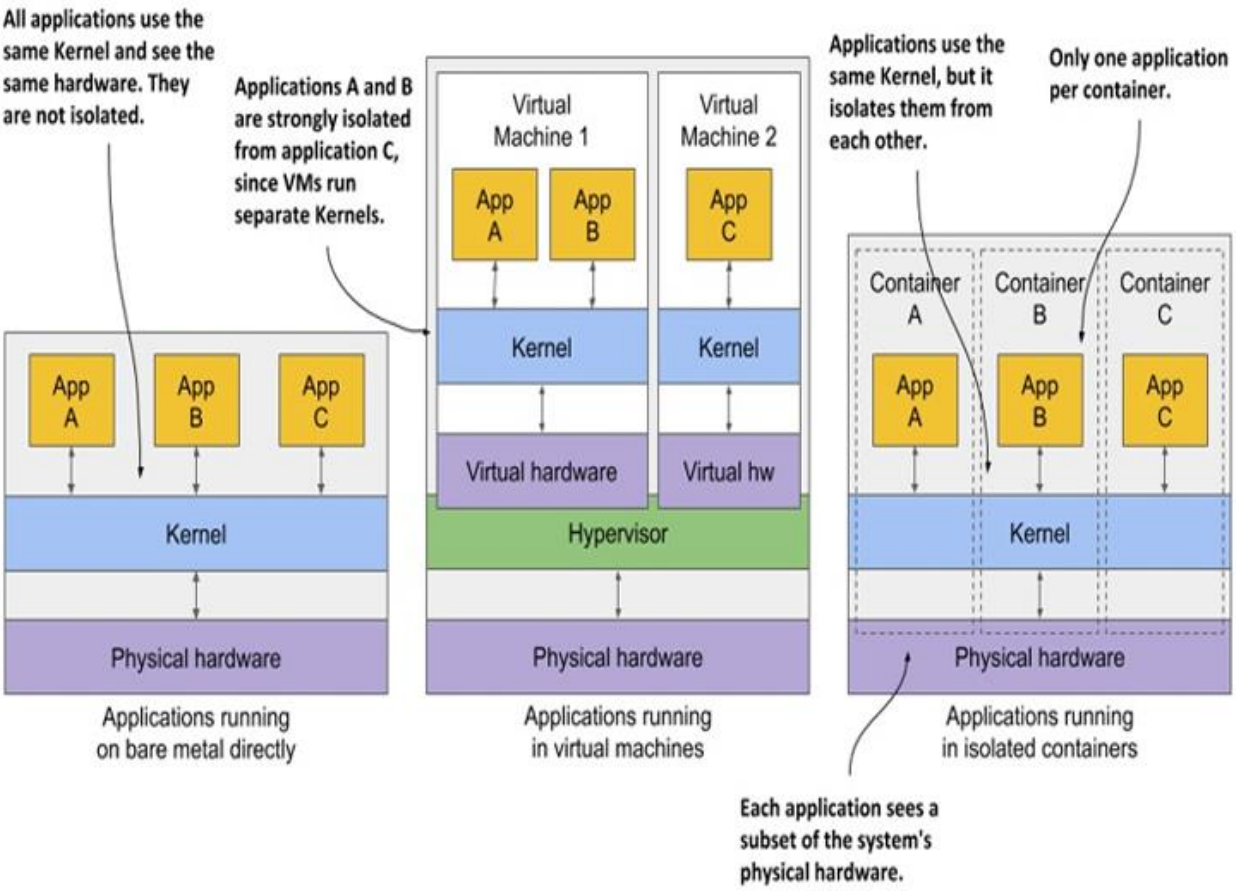
==========



==========


Virtual Machine
App
Kernel
Virtual hardware
Hypervisor
Physical hardware
Application in a VM
System calls
Container
App
Kernel
Physical hardware
Containerized application
Machine instructions

==========

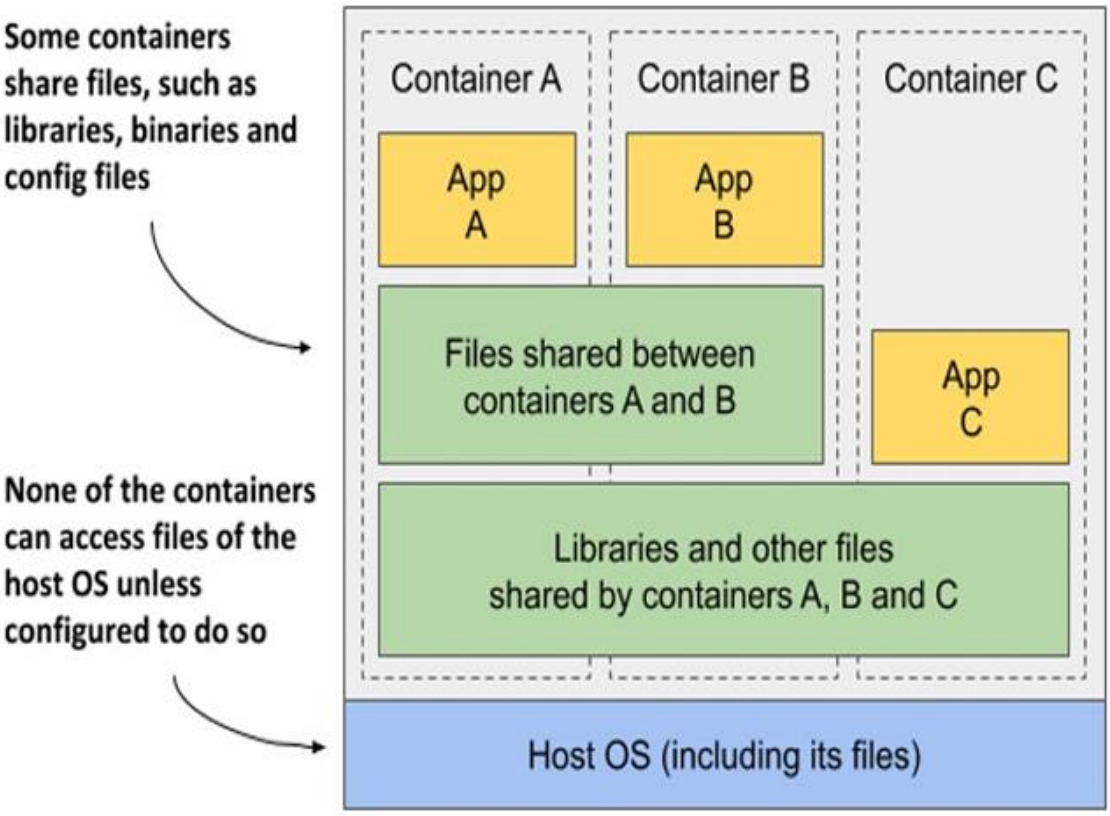


==========

Containers **share memory space**, whereas each VM **uses its own chunk of memory**. Therefore, if you don't limit the amount of memory that a container can use, this could cause other containers to run **out of memory** or cause their data to be **swapped out to disk**.

==========

Figure 2.8 Containers can share image layers



==========

The filesystems are isolated by the **Copy-on-Write** (**CoW**) mechanism. The filesystem of a container consists of **read-only layers** from the container image and an additional **read/write layer** stacked on top. When an application running in container A changes a file in one of the **read-only layers**, the entire file is copied into the container's **read/write layer,** and the file contents are changed there. Since each container **has its own writable layer**, changes to shared files are not visible in any other container.

==========

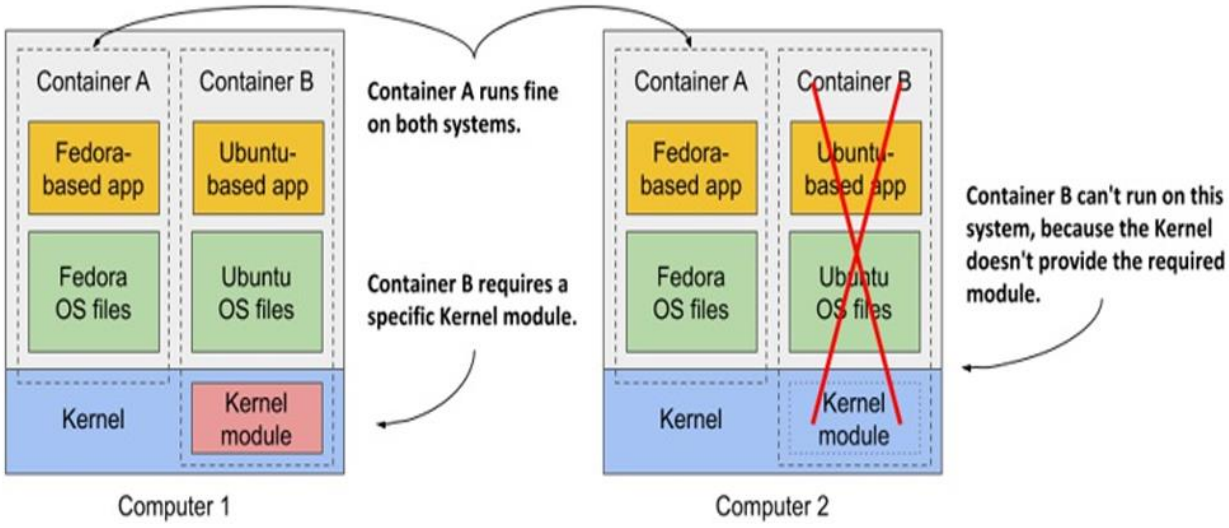
When you delete a file, it is only marked as deleted in the read/write layer, but it's still present in one or more of the layers below. **What follows is that deleting files never reduces the size of the image**.

==========

WARNING

Even seemingly harmless operations, such as changing permissions or ownership of a file, result in a new copy of the entire file being created in the read/write layer. If you perform this type of operation on a large file or many files, **the image size may swell significantly**.

==========



==========

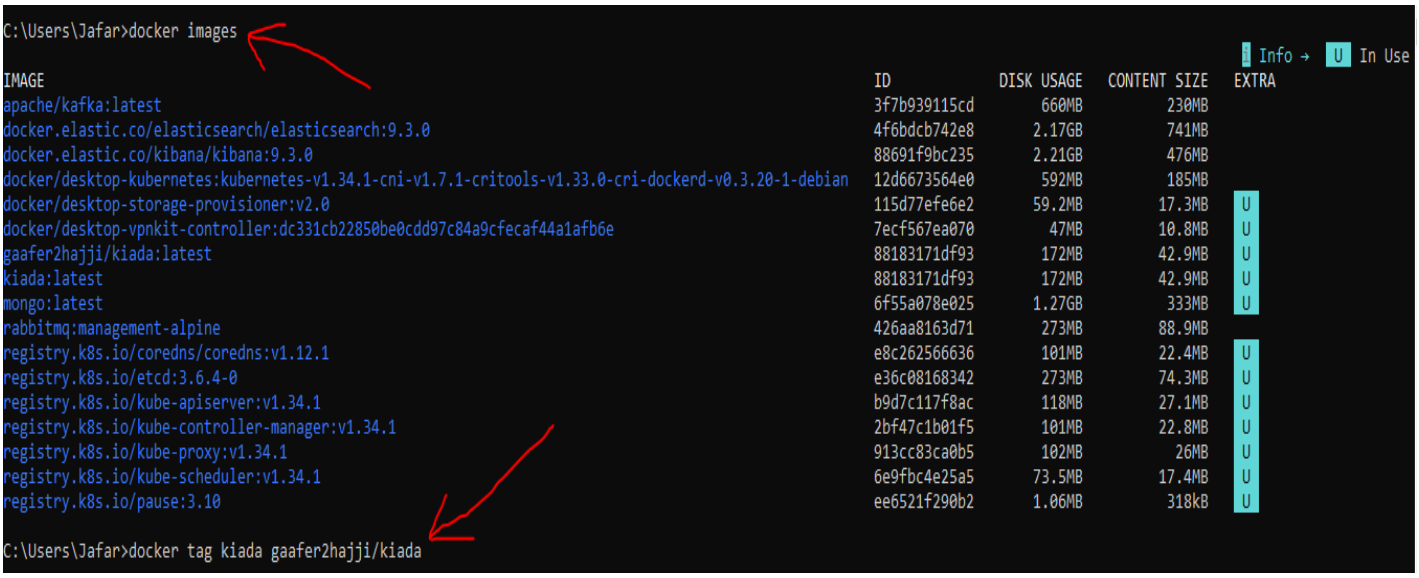
Getting additional information about a container

The `docker ps` command shows the most basic information about the containers. To see additional information, you can use `docker inspect`:

```
$ docker inspect kiada-container
```

Docker prints a long JSON-formatted document containing a lot of information about the container, such as its state, config, and network settings, including its IP address.

==========



==========

The first feature, **called Linux Namespaces,** ensures that each process has its own view of the system. This means that a process running in a container will only see some of the files, processes, and network interfaces on the system, as well as a different system hostname, **just as if it were running in a separate virtual machine**.

Initially, all the system resources available in a Linux OS, such as **filesystems**, **process IDs**, **user IDs**, **network interfaces**, and others, are all in the same bucket that all processes see and use. But the Kernel allows you to create additional buckets **known as namespaces** and move resources into them so that they are organized in smaller sets. **This allows you to make each set visible only to one process or a group of processes**. When you create a new process, you can specify which namespace it should use. The process only sees resources that are in this namespace and none in the other namespaces.

==========

Introducing the available namespace types

More specifically, there isn't just a single type of namespace. There are in fact several types – one for each resource type. A process thus uses not only one namespace, but one namespace for each type.

The following types of namespaces exist:

- The Mount namespace (mnt) isolates mount points (file systems).
- The Process ID namespace (pid) isolates process IDs.
- The Network namespace (net) isolates network devices, stacks, ports, etc.
- The Inter-process communication namespace (ipc) isolates the communication between processes (this includes isolating message queues, shared memory, and others).
- The UNIX Time-sharing System (UTS) namespace isolates the system hostname and the Network Information Service (NIS) domain name.
- The User ID namespace (user) isolates user and group IDs.
- The Time namespace allows each container to have its own offset to the system clocks.
- The Cgroup namespace isolates the Control Groups root directory.

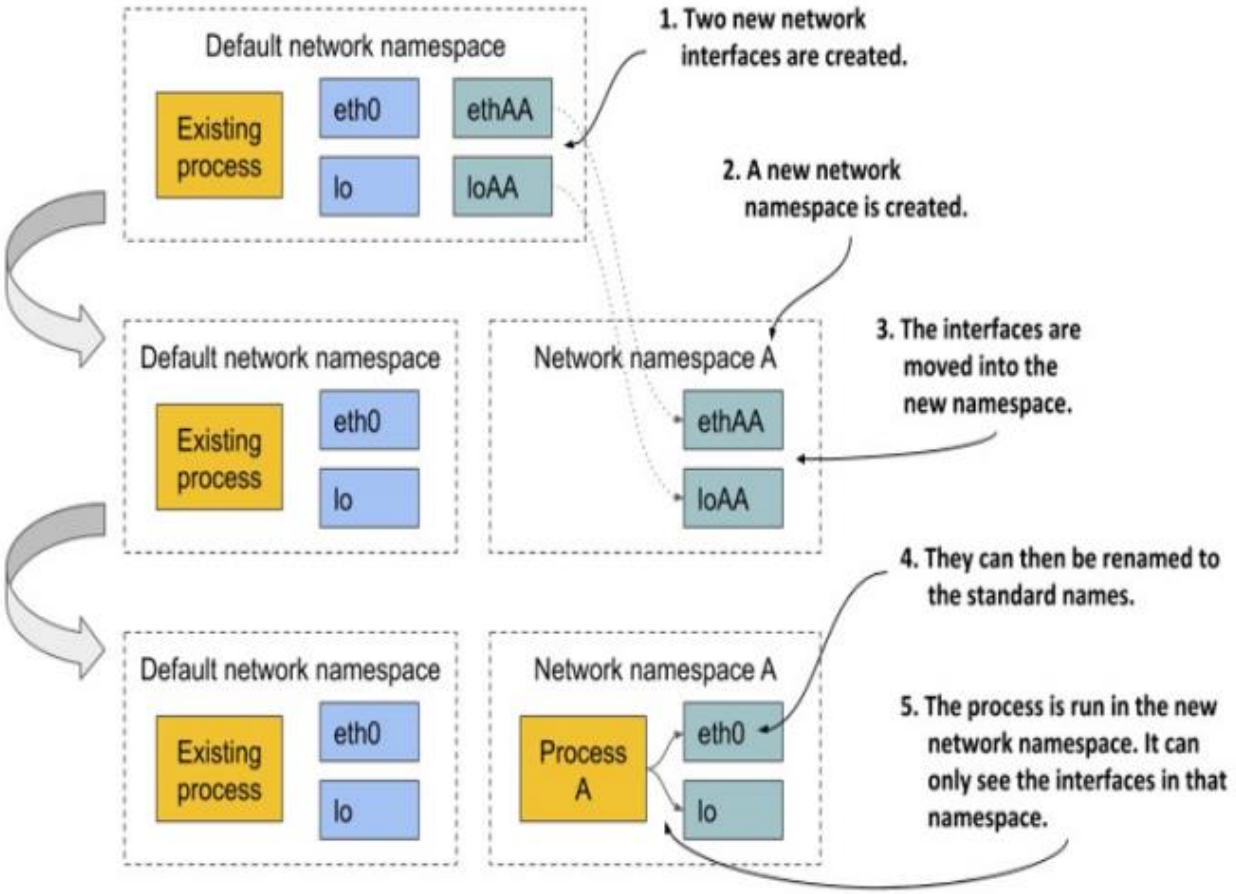
==========

Using network namespaces to give a process a dedicated set of network interfaces

The network namespace in which a process runs determines what network interfaces the process can see.

Each network interface belongs to exactly one namespace but can be moved from one namespace to another. If each container uses its own network namespace, **each container sees its own set of network interfaces**.

==========



==========

Initially, only the default network namespace exists. You then create two new network interfaces for the container and a new network namespace. The interfaces can then be moved from the default namespace to the new namespace. **Once there, they can be renamed**, because names **must only be unique in each namespace**. Finally, the process can be started in this network namespace, which allows it to only see the two interfaces that were created for it.

By looking solely at the available network interfaces, the process can't tell whether it's in a **container**, a **VM**, or an **OS** running directly **on a bare-metal machine**.

==========

Using the UTS namespace to give a process a dedicated hostname

Another example of how to make it look like the process is running on its own host is to use the UTS namespace. **It determines what hostname and domain name the process running inside this namespace sees**. By assigning two different UTS namespaces to two different processes, you can make them see different system hostnames. To the two processes, **it looks as if they run on two different computers**.

==========

Understanding how namespaces isolate processes from each other

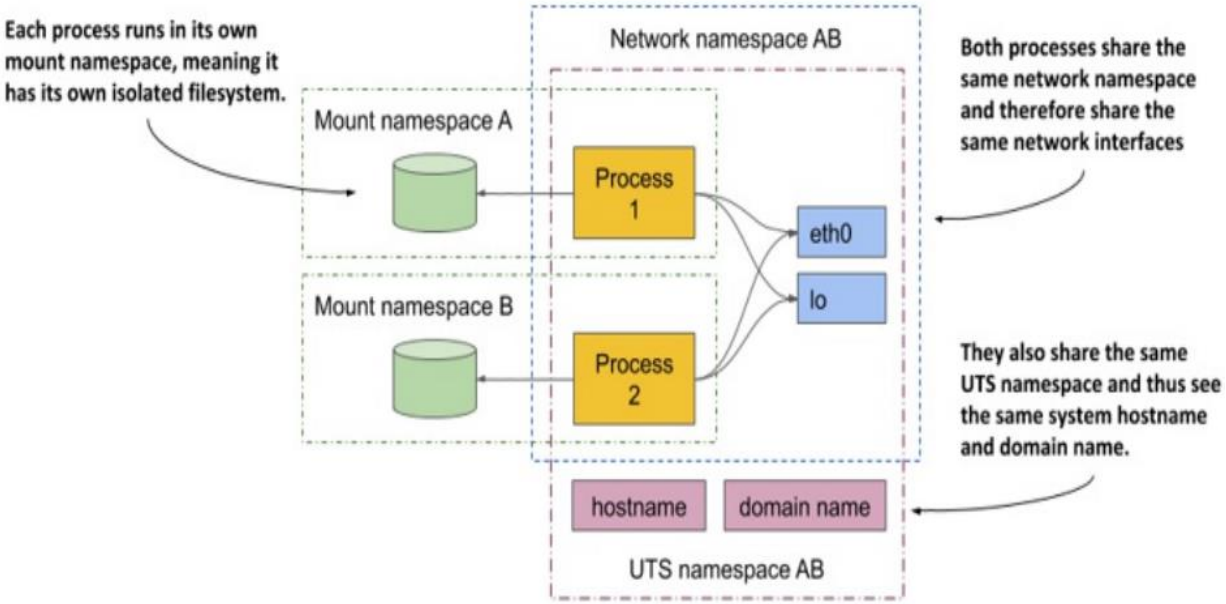
By creating a dedicated namespace instance for all available namespace types and assigning it to a process, you can make the process believe that it's running in its own OS. **The main reason for this is that each process has its own environment**. A process can only see and use the resources in its own namespaces. It can't use any in other namespaces. Likewise, **other processes can't use its resources either**. This is how containers isolate the environments of the processes that run within them.

==========

Sharing namespaces between multiple processes

We don't always want to isolate the containers completely from each other. **Related containers may want to share certain resources**.

==========


Each process runs in its own mount namespace, meaning it has its own isolated filesystem.
Network namespace AB
Mount namespace A
Process 1
eth0
lo
Mount namespace B
Process 2
Both processes share the same network namespace and therefore share the same network interfaces
They also share the same UTS namespace and thus see the same system hostname and domain name.
hostname
domain name
UTS namespace AB